



INT305 Machine Learning

Lecture 4

Support Vector Machine, SVM Loss and Softmax Loss

Sichen Liu

Department of Intelligence Science

Sichen.Liu@xjtlu.edu.cn

Binary Classification with a Linear Model

- Classification: Predict a discrete-valued target
- Binary classification: Targets $t \in \{-1, +1\}$
- Linear model:

$$z = \mathbf{w}^\top + b$$

$$y = \text{sign}(z)$$

- Question: How should we choose $\mathbf{w}$ and b ?

Zero-One Loss 0-1 loss

- We can use the 0-1 loss function, and find the weights that minimize it over data points

误差判断只有两种结果

与目标值相等为1

与目标值不相等为2

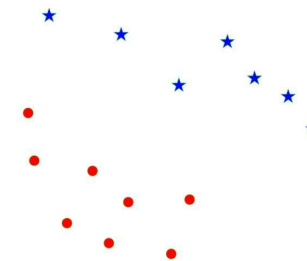
$$\mathcal{L}_{0-1}(y, t) = \begin{cases} 0 & \text{if } y = t \\ 1 & \text{if } y \neq t \end{cases}$$

$$= \mathbb{I}\{y \neq t\}$$

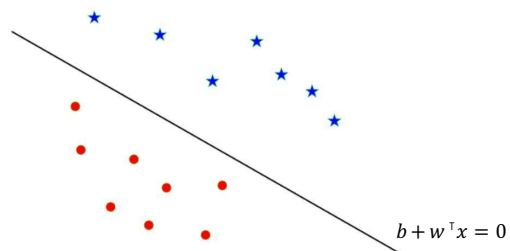
- But minimizing this loss is computationally difficult, and it can't distinguish different hypotheses that achieve the same accuracy. 但是因为这种方式得到的误差判断没有斜率
- We investigated some other loss functions that are easier to minimize, e.g., logistic regression with the cross-entropy loss $\mathcal{L}_{CE}$. 变化, 所以不能通过求导的方式计算
- Let's consider a different approach, starting from geometry of binary classifiers.

Separating Hyperplanes

Suppose we are given these data points from two different classes and want to find a linear classifier that separates them.

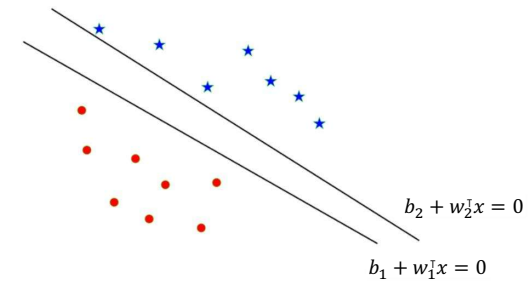


Separating Hyperplanes



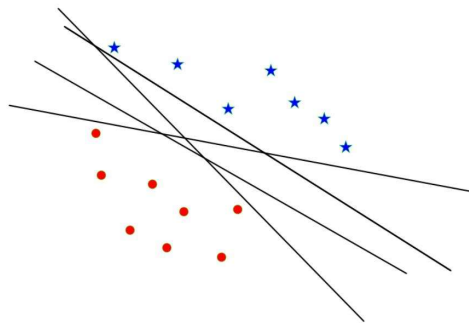
- The decision boundary looks like a line because $\mathbf{x} \in \mathbb{R}^2$, but think about it as a $D - 1$ dimensional hyperplane.
- Recall that a hyperplane is described by points $\mathbf{x} \in \mathbb{R}^D$ such that $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b = 0$.

Separating Hyperplanes



- There are multiple separating hyperplanes, described by different parameters $(\mathbf{w}, b)$.
有无数个不同的 $(\mathbf{w}, b)$ 组成结果。

Separating Hyperplanes



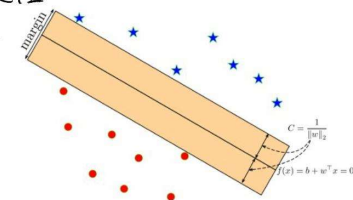
Optimal Separating Hyperplane

Optimal Separating Hyperplane: A hyperplane that separates two classes and maximizes the distance to the closest point from either class, i.e., maximize the **margin** of the classifier.

我们通常更想找到最优超平面

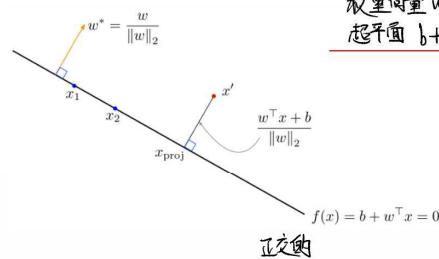
边界距离两个类别最近的点
的距离保持尽可能远。

不偏向任何一方



Intuitively, ensuring that a classifier is not too close to any data points leads to better generalization on the test data.

Geometry of Points and Planes 理解一个点到一个平面的距离是怎么算的



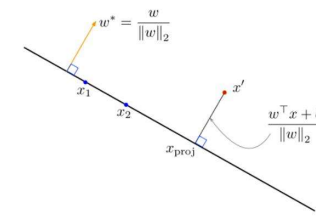
权重向量 w 的方向，永远垂直于决策超平面 $b + w^T x = 0$

正交的

- Recall that the decision hyperplane is orthogonal (perpendicular) to w .
- The vector $w^* = \frac{w}{\|w\|_2}$ is a unit vector pointing in the same direction as w .
- The same hyperplane could equivalently be defined in terms of w^* .

单位向量 w^* ，向量 w 的长度 $\|w\|_2$ 是可以任意变化的。为了方便几何计算，经常会把 w 标准化，也就是把它变成长度为1的单位向量 w^* 。 $\frac{w}{\|w\|_2}$ 长度固定为1。

Geometry of Points and Planes



The (signed) distance of a point x' to the hyperplane is

$$\frac{w^T x' + b}{\|w\|_2} = d$$

$\|w\|_2$ 代表 w 的标量长度

目标：计算一个点 x' 到超平面 $w^T x + b = 0$ 的垂直距离 d 。

几何分析：

1. 我们在超平面上找到距离 x' 最近的那个点，称为 x_{proj} (x' 在平面上的投影点。) 距离 d 的长度为 $\|x' - x_{proj}\|_2$ 。

2. $(x' - x_{proj})$ 的方向垂直于超平面，所以和平面上的向量 w 是平行的。

3. 利用向量投影来计算这个距离。

在超平面上选任意一个点，称为 x_0 。

我们构造一个从平面指向 x' 的向量 $u = (x' - x_0)$ 。

距离 $d =$ 向量 u 在法向量 w 上的投影长度。

4. 根据数学投影公式 u 在 w 上的投影长度为

$$d = \frac{|u \cdot w|}{\|w\|_2}$$

Maximizing Margin as an Optimization Problem

- Recall: the classification for the i -th data point is correct when

$$\text{sign}(\mathbf{w}^T \mathbf{x}^{(i)} + b) = t^{(i)}$$

- This can be rewritten as

$$t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) > 0$$

- Enforcing a margin of C :

$$t^{(i)} \cdot \underbrace{\frac{(\mathbf{w}^T \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|_2}}_{\text{signed distance}} \geq C$$

要确保几何间隔大于 C .

除了要正确分类, 如果点离边界很近, 虽然分类对了, 但风险依旧很高, 所以需要尽可能这个点距离决策边界有一定的距离. 这段几何间隔记为 γ .

Maximizing Margin as an Optimization Problem

Max-margin objective:

$$\max_{\mathbf{w}, b} C$$

$$\text{s. t. } \frac{t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|_2} \geq C \quad i = 1, \dots, N$$

Plug in $C = 1/\|\mathbf{w}\|_2$ and simplify:

$$\underbrace{\frac{t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|_2} \geq \frac{1}{\|\mathbf{w}\|_2}}_{\text{geometric margin constraint}} \iff \underbrace{t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1}_{\text{algebraic margin constraint}}$$

Equivalent optimization objective:

$$\min \|\mathbf{w}\|_2^2$$

$$\text{s. t. } t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 \quad i = 1, \dots, N$$

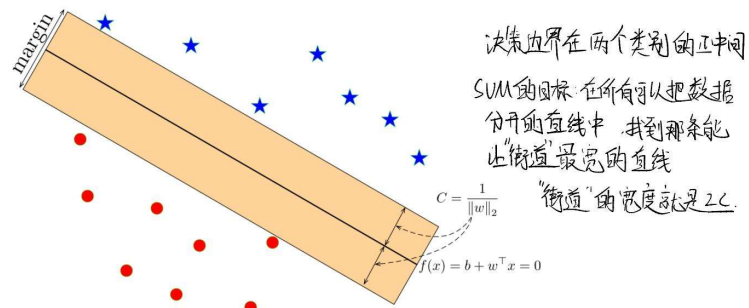
为了方便计算, 对于 $\mathbf{w}$ 和 b , 我们可以进行任意的缩放, 而决策边界的位置不变. 利用这个特性, 我们假设刚好在决策边界线上的点的分数为 1 ($\mathbf{w}^T \mathbf{x} + b = 1$), 也就是决策边界单侧的宽度.

是 $\frac{1}{\|\mathbf{w}\|}$, $C = \frac{1}{\|\mathbf{w}\|}$

这样, 最大化 C 的目标, 就变成最小化 $\|\mathbf{w}\|$. 为求导方便, 通常基于 $\|\mathbf{w}\|^2$ 来最小化.

Maximizing Margin as an Optimization Problem

SVM相较于逻辑回归的优势:

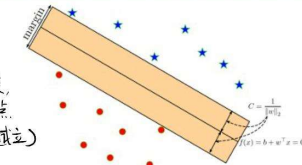


Maximizing Margin as an Optimization Problem

Algebraic max-margin objective:

最小化 $\min \|w\|_2^2$
 约束条件 s.t. $t^{(i)}(w^T x^{(i)} + b) \geq 1 \quad i = 1, \dots, N$

(硬分类, 对所有点都必须成立)



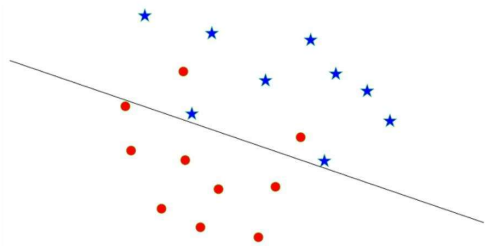
- Observe: if the margin constraint is not tight for $x^{(i)}$, we could remove it from the training set and the optimal w would be the same.
- The important training examples are the ones with algebraic margin 1, and are called **support vectors**.
- Hence, this algorithm is called the (hard) **Support Vector Machine (SVM)** (or Support Vector Classifier).
- SVM-like algorithms are often called **max-margin** or **large-margin**.

对于硬间隔 SVM, 最终计算中的 margin 宽度完全由支持向量决定 (因为生成于决策边界两端的点), 即满足 $t(w^T x + b) = 1$, 其他间隔更大的点都无关紧要, 即使去掉也不会影响结果

硬间隔必须要正确分类所有点, 如果存在噪声点则分类会失效.

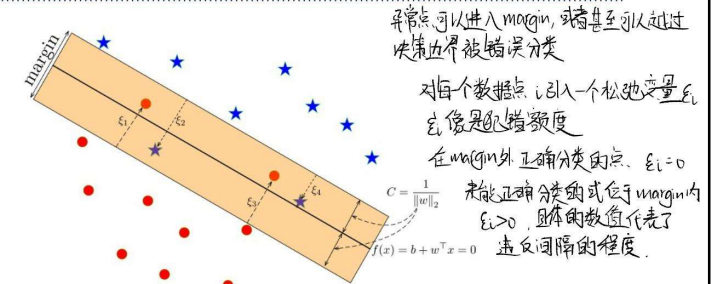
Non-Separable Data Points

How can we apply the max-margin principle if the data are **not** linearly separable?



对于存在异常点的情况, 只能考虑利用软间隔 SVM.

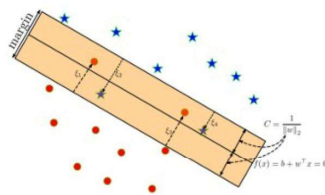
Maximizing Margin for Non-Separable Data Points



Main idea:

- Allow some points to be within the margin or even be misclassified; we represent this with slack variables ϵ_i .
- But constrain or penalize the total amount of slack.

Maximizing Margin for Non-Separable Data Points



- Soft margin constraint:

$$\frac{t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b)}{\|\mathbf{w}\|_2} \geq C(1 - \xi_i),$$

for $\xi_i \geq 0$.

- Penalize $\sum_i \xi_i$.

如果 $\xi_i = 0$, 则等效于 hard SVM.
 如果 $0 < \xi_i \leq 1$, 这个点进入间隔, 但在正确的一侧.
 如果 $\xi_i > 1$, 这个点已经越过了决策边界, 被彻底错误分类.

Maximizing Margin for Non-Separable Data Points

Soft-margin SVM Objective:

$$\begin{aligned} \text{最小化} \quad & \min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|_2^2 + \gamma \sum_{i=1}^N \xi_i && \frac{1}{2} \|\mathbf{w}\|_2^2 \text{ 让其最小化是为了让间隔足够宽,} \\ \text{约束条件} \quad & \text{s.t. } t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \xi_i \quad i = 1, \dots, N && \gamma \geq \xi_i \text{ 新的惩罚项,} \\ & \xi_i \geq 0 && \text{让所有点都错的} \\ & && \text{总惩罚尽可能小.} \end{aligned}$$

- γ is a hyperparameter that trades off the margin with the amount of slack.

- For $\gamma = 0$, we'll get $\mathbf{w} = 0$. (why?)
- As $\gamma \rightarrow \infty$, we get the hard-margin objective.

- Note: it is also possible to constrain $\sum_i \xi_i$ instead of penalizing it.

γ 是一个需要我们人为定义参数.
 • 如果 γ 很大: 意味着我们对犯错的惩罚极重, 模型会非常努力地正确分类每一个点, 宁可选择一个很窄的间隔. 当 γ 趋向无穷大时, 软间隔就变成了硬间隔.
 • 如果 γ 很小: 意味着我们对犯错比较宽容, 模型会更倾向于找到一个更宽, 更简单的间隔, 哪怕牺牲掉几个点的正确分类.

将带约束的软间隔SVM问题, 转换成-一个更通用的, 无约束的“损失函数+正则化”形式.

From Margin Violation to Hinge Loss 合页损失

Let's simplify the soft margin constraint by eliminating ξ_i . Recall:

$$t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 - \xi_i \quad i = 1, \dots, N$$

$$\xi_i \geq 0 \quad i = 1, \dots, N$$

- Rewrite as $\xi_i \geq 1 - t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b)$. Case 1:
- Case1:** $1 - t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 0$ 点“守规矩”. 如果一个点被正确分类, 并且在间隔之外, 那么 $t(\mathbf{w}^T \mathbf{x} + b) \geq 1$.
 > The smallest non-negative ξ_i that satisfies the constraint is $\xi_i = 0$. 此时 $1 - t(\mathbf{w}^T \mathbf{x} + b) \leq 0$, 那么 ξ_i 就要大于等于一个 ≤ 0 的数, 同时 $\xi_i \geq 0$.
- Case2:** $1 - t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) < 0$
 > The smallest ξ_i that satisfies the constraint is $\xi_i = 1 - t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b)$. 因为总目标是最小化 $\sum \xi_i$, 则满足上述条件的情况为 $\xi_i = 0$.
- Hence, $\xi_i = \max\{0, 1 - t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b)\}$.
- Therefore, the slack penalty can be written as

$$\sum_{i=1}^N \xi_i = \sum_{i=1}^N \max\{0, 1 - t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b)\}$$

Case 2: 点“不守规矩”:
 如果一个点在间隔之内被错分, 那么它的分数 $t(\mathbf{w}^T \mathbf{x} + b) < 1$.
 $1 - t(\mathbf{w}^T \mathbf{x} + b) > 0$.
 对 ξ_i 的要求是 $\xi_i \geq$ 某个 > 0 的数.
 为了最小化 ξ_i , 我们会取最小的临界值, 也就是 $\xi_i = 1 - t(\mathbf{w}^T \mathbf{x} + b)$.

From Margin Violation to Hinge Loss

把结果代入 $\min_{\mathbf{w}, b, \xi} \|\mathbf{w}\|_2^2 + \gamma \sum_{i=1}^N \xi_i$

If we write $y^{(i)}(\mathbf{w}, b) = \mathbf{w}^T \mathbf{x} + b$, then the optimization problem can be written as

$$\text{替换后: } \min_{\mathbf{w}, b, \xi} \sum_{i=1}^N \max\{0, 1 - t^{(i)}y^{(i)}(\mathbf{w}, b)\} + \frac{1}{2\gamma} \|\mathbf{w}\|_2^2 \quad y(\mathbf{w}, b) \text{ 就是分数 } \mathbf{w}^T \mathbf{x} + b.$$

- The loss function $\mathcal{L}_H(y, t) = \max\{0, 1 - ty\}$ is called the **hinge loss**. 合页损失. 如果分数 $ty \geq 1$ (即正确分类且间隔外), 那么损失为 0, 模型满意.
- The second term is the L_2 -norm of the weights. 如果分数 $ty < 1$ (违反了间隔), 那么损失是 $1 - ty$, 是一个自归一化解.
- Hence, the soft-margin SVM can be seen as a linear classifier with hinge loss and an L_2 linear growth regularizer. 惩罚.

后面的 $\|\mathbf{w}\|_2^2$ 被称为 L_2 正则项, 作用是惩罚过于复杂的模型.
 在 SVM 的语境下, 最小化 $\|\mathbf{w}\|_2^2$ 等价于最大化间隔.

MulticlassSVMLoss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1

$s_j - s_{y_i} + 1$ 就是在对比这两个分数。只有当这个值大于0才会有损失

$s_j - s_{y_i} + 1 > 0$ 等于 $s_{y_i} < s_j + 1$ 。这意味着正确类别的分数没有比这个错误类别的分数高出至少1，那么模型会受到惩罚，惩罚的大小取决于这个安全间隔目标的差距。

$\sum_j \max(0, s_j - s_{y_i} + 1)$: 要把所有的错误类别的情况都检查一遍，然后把他们的损失加起来

$\max(0, s_j - s_{y_i} + 1)$: 针对某一个错误类别的损失计算

s_j : 是这个错误类别的分数

s_{y_i} : 是正确类别的分数

MulticlassSVMLoss:

Given an example (x_i, y_i) , where x_i is the image and y_i is the (integer) label.

and using the shorthand for the scores vector: $s = f(x_i, W)$

The SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

公式含义: 我们希望正确类别的分数, 应该比任何一个错误类别的分数都至少高出一个“安全间隔”(这里设置为1)

MulticlassSVMLoss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9		

样本 i , 类别 j

MulticlassSVMLoss:

Given an example (x_i, y_i) , where x_i is the image and y_i is the (integer) label.
and using the shorthand for the scores vector: $s = f(x_i, W)$

The SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$\begin{aligned} &= \max(0, 5.1 - 3.2 + 1) \\ &\quad + \max(0, -1.7 - 3.2 + 1) \\ &= \max(0, 2.9) + \max(0, -3.9) \\ &= 2.9 + 0 \\ &= 2.9 \end{aligned}$$

计算猫的损失

$\max(0, 2.9)$ 猫与车之间的损失, 车好猫
 $\max(0, -3.9)$ 猫与青蛙之间的损失, 猫的分
数没有比青蛙的分数高出安全间隔

MulticlassSVMLoss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:

			
cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	

SVM与其他分类器的区别:
例如 softmax 的目标是无限接近完美, 希望正确类别的概率有100%。
SVM 目标更务实: 不求完美, 只求正确类别的分数能比一个完全间隔战胜所有错误分数。一个点达到了这个分类目标, SVM 就会将注意力转到还在间隔内或者被错分的训练样本上。

MulticlassSVMLoss:

Given an example (x_i, y_i) ,
where x_i is the image and y_i is the (integer) label.
and using the shorthand for the scores vector: $s = f(x_i, W)$

The SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$
$$= \max(0, 1.3 - 4.9 + 1) + \max(0, 2.0 - 4.9 + 1)$$
$$= \max(0, -2.6) + \max(0, -1.9)$$
$$= 0 + 0$$
$$= 0$$

损失值为0, 模型认为当前权重 W 已经足够好了, 在接下来梯度下降中, 这张汽车图片不会对权重更新做出任何新贡献

MulticlassSVMLoss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:

			
cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

MulticlassSVMLoss:

Given an example (x_i, y_i) ,
where x_i is the image and y_i is the (integer) label.
and using the shorthand for the scores vector: $s = f(x_i, W)$

The SVM loss has the form:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$
$$= \max(0, 2.2 - (-3.1) + 1) + \max(0, 2.5 - (-3.1) + 1)$$
$$= \max(0, 6.3) + \max(0, 6.6)$$
$$= 6.3 + 6.6$$
$$= 12.9$$

MulticlassSVMLoss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:

			
cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

MulticlassSVMloss:
Given an example (x_i, y_i) ,
where x_i is the image and y_i is the
(integer) label.
and using the shorthand for the scores
vector: $s = f(x_i, W)$
The SVM loss has the form:
$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

and the full training loss is the mean over
all examples in the training data:
$$L = \frac{1}{N} \sum_{i=1}^N L_i$$

$$L = (2.9 + 0 + 12.9)/3$$

$$= 5.27$$

总的训练损失是所有训练损失相加求平均值。

MulticlassSVMLoss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:

			
cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

MulticlassSVMloss:
Given an example (x_i, y_i) ,
where x_i is the image and y_i is the
(integer) label.
and using the shorthand for the scores
vector: $s = f(x_i, W)$
The SVM loss has the form:
$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

**Q: what if the sum was instead over all
classes? (including $j=y_i$)**
公式原来只关注不等于当前类别的其他错误类别来求和。
如果对所有类别求和，则给出自身本身的损失。
$$\max(0, s_{y_i} - s_{y_i} + 1) = 1$$

添加一个常数不改变任何事情。

MulticlassSVMLoss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

MulticlassSVMloss:
Given an example (x_i, y_i) ,
where x_i is the image and y_i is the
(integer) label.
and using the shorthand for the scores
vector: $s = f(x_i, W)$

The SVM loss has the form:
$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Q2: what if we used a mean instead of a
sum here

新公式: $L_i = \frac{1}{k-1} \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$
k是类别数, k-1指排除类别数量
新公式相当于给整个损失函数乘了一个正常数 $(\frac{1}{k-1})$
不会变化

MulticlassSVMLoss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

MulticlassSVMloss:
Given an example (x_i, y_i) ,
where x_i is the image and y_i is the
(integer) label.
and using the shorthand for the scores
vector: $s = f(x_i, W)$

The SVM loss has the form:
$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Q3: what if we used

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)^2$$

这里主要改变了对错误的惩罚方式

- 标准负损失: 惩罚是线性增长的
- 平方负损失: 惩罚是平方增长的

大的异常样本会更加敏感

$s_j - s_{y_i} + 1$ 结果小的异常样本最终会更接近0,
从而被忽略

MulticlassSVMLoss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

MulticlassSVMloss:
Given an example (x_i, y_i) ,
where x_i is the image and y_i is the
(integer) label.
and using the shorthand for the scores
vector: $s = f(x_i, W)$

The SVM loss has the form:
$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Q4: what is the min/max possible loss

最小是0
最大是∞

MulticlassSVMLoss

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



类别	cat	3.2	1.3	2.2
样本	car	5.1	4.9	2.5
	frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9	

假设有个类别, 那么对每个样本来说
 $L_i = (C-1) \times 1 = C-1$
如果数据集有N个样本, 总损失为
 $L = \sum_{i=1}^N L_i = N \cdot (C-1)$

MulticlassSVMloss:
Given an example (x_i, y_i) ,
where x_i is the image and y_i is the
(integer) label.
and using the shorthand for the scores
vector: $s = f(x_i, W)$

The SVM loss has the form:
$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

Q5: usually at initialization W are small
numbers so all $s \approx 0$. What is the loss?

$s_j \approx 0, s_{y_i} \approx 1$
 $L_i = \sum_{j \neq y_i} \max(0, 0 - 0 + 1)$
 $= \sum_{j \neq y_i} \max(0, 1) = \sum_{j \neq y_i} 1$
有k-1个类别, 所以最开始每个样本损失k-1

Softmax

SoftmaxClassifier(Multinomial Logistic Regression)



cat	3.2
car	5.1
frog	-1.7

Softmax

Softmax的前期流程和SVM类似, 是基于输入图片和权重w计算分数

SoftmaxClassifier(Multinomial Logistic Regression)



scores=unnormalizedlogprobabilitiesoftheclasses.

$$s = f(x_i; W)$$

cat	3.2
car	5.1
frog	-1.7

Softmax

SoftmaxClassifier(Multinomial Logistic Regression)

模型对于输入属于不同类别的偏好程度,分数越高,偏好越强.

归一化的对数概率.

scores=unnormalized log probabilities of the classes.



$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}} \text{ where } s = f(x_i; W)$$

cat	3.2
car	5.1
frog	-1.7

Softmax

SoftmaxClassifier(Multinomial Logistic Regression)



scores=unnormalized log probabilities of the classes.

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}} \text{ where } s = f(x_i; W)$$

cat	3.2
car	5.1
frog	-1.7

Softmax function

为了让分数有意义,我们需要将它们转换成一个概率分布

1. 指数化, 对每一个分数取指数 e^s

例: $e^{3.2} \approx 24.5$ $e^{5.1} \approx 164.0$ $e^{-1.7} \approx 0.18$

2. 归一化: 将上面得到的三个正数相加, 得到一个总和

$24.5 + 164.0 + 0.18 = 188.68$

$P(\text{cat}) = \frac{24.5}{188.68} \approx 0.13 \Rightarrow 13\%$ 87% 把握是车

$P(\text{car}) = \frac{164.0}{188.68} \approx 0.87 \Rightarrow 87\%$

$P(\text{frog}) = \frac{0.18}{188.68} \approx 0.00 \Rightarrow \text{approx } 0\%$

Softmax

SoftmaxClassifier(Multinomial Logistic Regression)



scores=unnormalized log probabilities of the classes.

$$P(Y = k | X = x_i) = \frac{e^{s_k}}{\sum_j e^{s_j}} \text{ where } s = f(x_i; W)$$

cat	3.2
car	5.1
frog	-1.7

Want to maximize the log likelihood, or (for a loss function) to minimize the negative log likelihood of the correct class:

$$L_i = -\log P(Y = y_i | X = x_i)$$

Softmax

SoftmaxClassifier(Multinomial Logistic Regression)



cat	3.2
car	5.1
frog	-1.7

unnormalized log probabilities

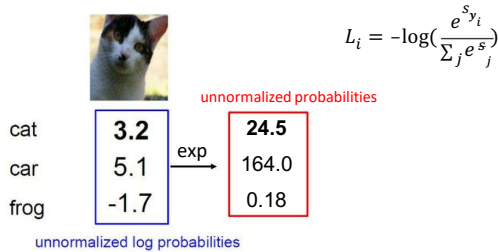
$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

使用交叉熵来计算损失。
目标：我们希望模型为正确类别预测出的概率尽可能高。
损失函数：最大化正确类别的概率，等价于最小化正确类别的负对数概率。

$L_i = -\log(0.13)$
因为正确类别的概率低，所以对数的结果大，代表了大的损失。

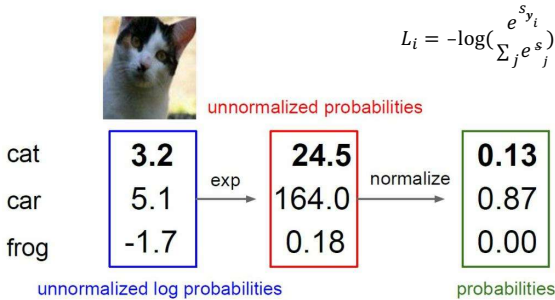
Softmax

SoftmaxClassifier(Multinomial Logistic Regression)



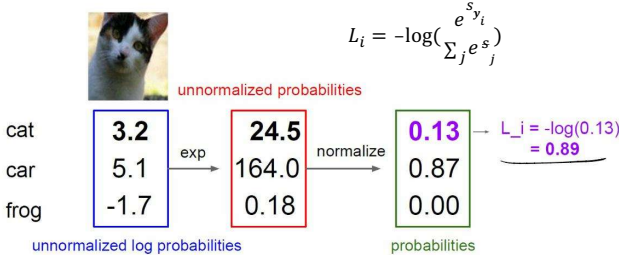
Softmax

SoftmaxClassifier(Multinomial Logistic Regression)



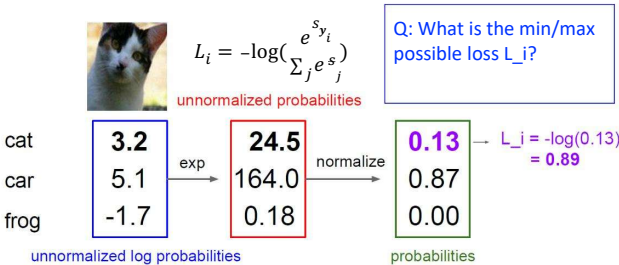
Softmax

SoftmaxClassifier(Multinomial Logistic Regression)



Softmax

SoftmaxClassifier(Multinomial Logistic Regression)



Softmax

SoftmaxClassifier(Multinomial Logistic Regression)



$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

unnormalized probabilities

cat
car
frog

3.2
5.1
-1.7

unnormalized log probabilities

24.5
164.0
0.18

exp

normalize

0.13
0.87
0.00

probabilities

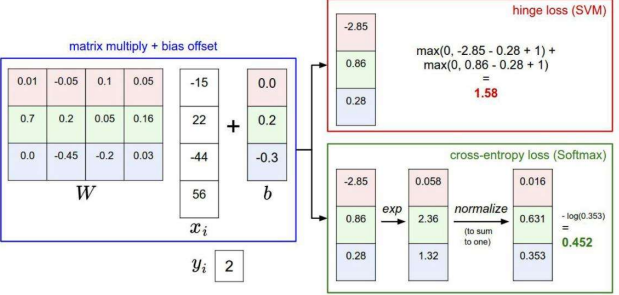
Q5: usually at initialization
W are small numbers, so
all s ~ 0. What is the loss?

$$L_i = -\log(0.13)$$

= 0.89

Log(类的数量)
如果有3类, 就是 log(3).

SVM&Softmax



SVM&Softmax

Softmax vs. SVM

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

$$L = \sum_i \max_{j \neq y_i} (0, s_j - s_{y_i} + 1)$$

SVM&Softmax

Softmax vs. SVM

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

$$L = \sum_i \max_{j \neq y_i} (0, s_j - s_{y_i} + 1)$$

assume scores:

[10, -2, 3]

[10, 9, 9] ←

[10, -100, -100]

and $y_i = 0$

Q: suppose I take a datapoint and I jiggle a bit (changing its score slightly). What happens to the loss in both cases?

不是很确定的概率。

$$-\log(0.6) \approx 0.2$$

SVM会持续更新w, 确保正确类别的概率很有说服力。